SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	N Goutham	**Reg. No.:**	192472024
Section / Batch:	CSE-AI	**Date:**	

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins
- Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
  Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

9. FAISS Top-K Retrieval Analysis
Develop a FAISS semantic-search system and experiment with:
Top-1, Top-3, Top-5, Top-10 and Top-20

retrieval. Compare the retrieved documents, search latency, and relevance. Analyze the effect of increasing k and recommend an appropriate value for a semantic-search application.

Q9. FAISS Top-K Retrieval Analysis

1. Aim

To develop a semantic-search system using FAISS and analyze the effect of different Top-K values on document retrieval performance, search latency, and retrieval relevance.

2. Objective

The objective of this experiment is to evaluate FAISS semantic search using different values of K, namely Top-1, Top-3, Top-5, Top-10, and Top-20.

The experiment compares the documents retrieved at different K values, measures search latency, evaluates retrieval relevance using multiple queries, and identifies an appropriate K value for a semantic-search application.

3. Technologies Used

- Python
- Google Colab
- FAISS
- Sentence Transformers
- all-MiniLM-L6-v2
- NumPy
- Pandas
- Matplotlib

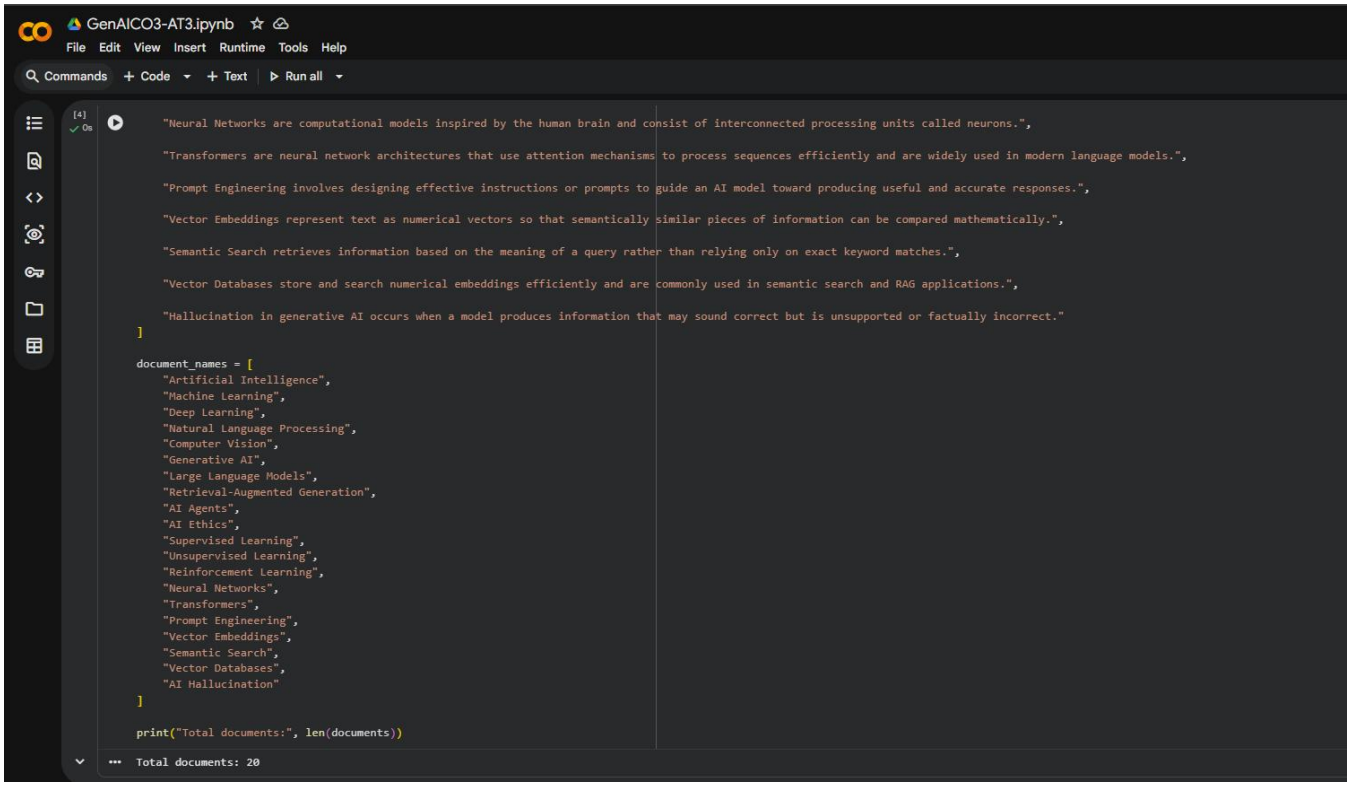
4. Dataset / Document Collection

A collection of 20 AI-related documents was created for the experiment. The documents cover topics including:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Natural Language Processing
- Computer Vision
- Generative AI
- Large Language Models
- Retrieval-Augmented Generation
- AI Agents
- AI Ethics
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning
- Neural Networks
- Transformers
- Prompt Engineering

- Vector Embeddings
- Semantic Search
- Vector Databases
- AI Hallucination

These documents were used as the knowledge collection for FAISS semantic retrieval.

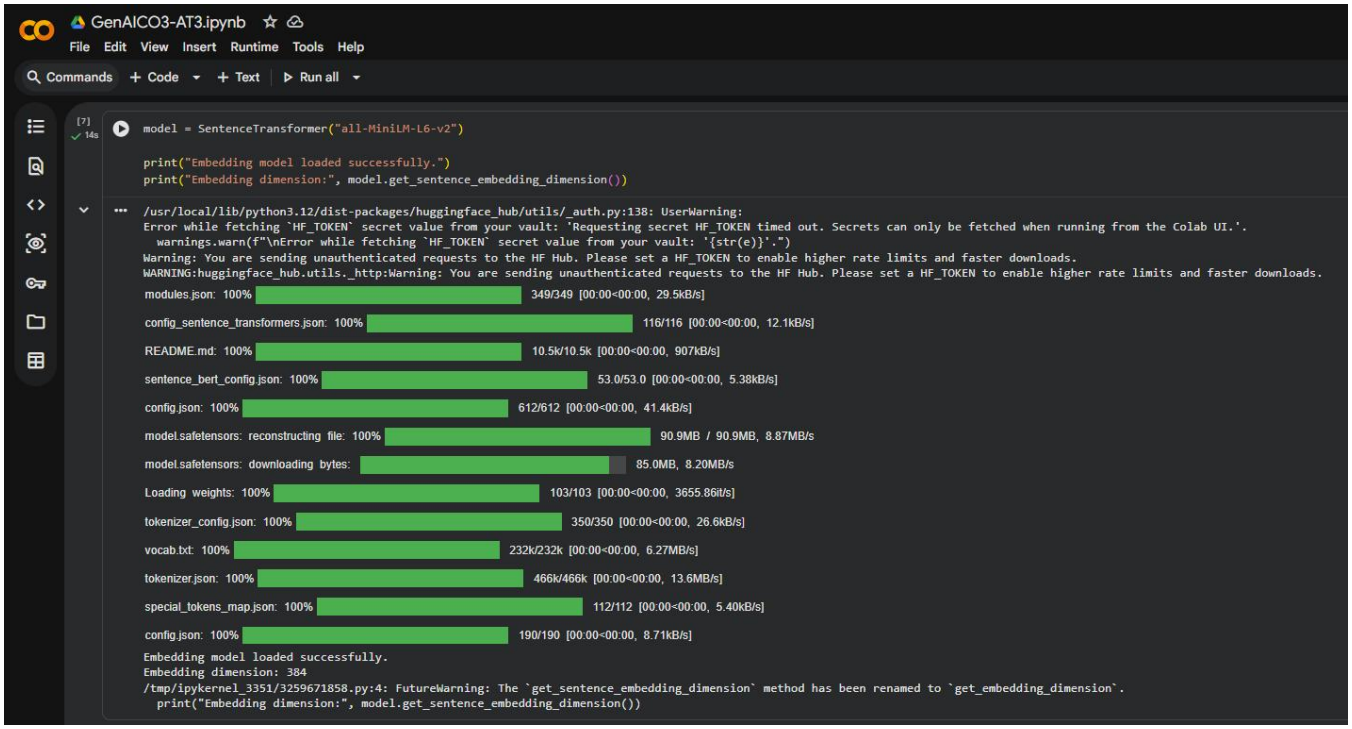


5. Embedding Generation

The all-MiniLM-L6-v2 Sentence Transformer model was used to convert the documents into numerical vector representations.

The embedding model generated vectors with a dimension of 384. A total of 20 document embeddings were generated.

These vector representations allow FAISS to compare the semantic similarity between a user query and the stored documents.



6. FAISS Index Creation

FAISS IndexFlatL2 was used to store and search the document embeddings.

The index uses L2/Euclidean distance to determine how close the query embedding is to each document embedding.

The FAISS index contained 20 vectors, with each vector having 384 dimensions.

A lower distance indicates that the document embedding is closer to the query embedding and is therefore considered more semantically relevant.

```
document_embeddings = model.encode(
    documents,
    convert_to_numpy=True
)

document_embeddings = document_embeddings.astype("float32")

print("Embedding generation completed.")
print("Number of documents:", len(document_embeddings))
print("Embedding shape:", document_embeddings.shape)
```

```
Embedding generation completed.
Number of documents: 20
Embedding shape: (20, 384)
```

7. Semantic Search Process

When a query is entered, it is converted into an embedding using the same Sentence Transformer model.

The query embedding is then passed to the FAISS index. FAISS calculates the distance between the query vector and stored document vectors and returns the nearest documents.

The retrieval process is:

User Query → Query Embedding → FAISS Search → Distance Calculation → Top-K Documents

```
#6#
dimension = document_embeddings.shape[1]

faiss_index = faiss.IndexFlatL2(dimension)

faiss_index.add(document_embeddings)

print("FAISS index created successfully.")
print("Vector dimension:", dimension)
print("Total vectors stored:", faiss_index.ntotal)
```

```
FAISS index created successfully.
Vector dimension: 384
Total vectors stored: 20
```

8. Top-K Retrieval Experiment

The experiment was performed using the following K values:

K = 1, 3, 5, 10, and 20

For each K value, FAISS returned the corresponding number of most relevant documents.

Top-1

Top-1 returns only the single most relevant document. It provides a highly focused result and minimizes the amount of retrieved information.

Top-3

Top-3 returns the three closest documents. It provides additional context while still keeping the result set relatively small.

Top-5

Top-5 returns five relevant documents and provides a balance between focused retrieval and information coverage.

Top-10

Top-10 provides a wider range of retrieved documents. It increases information coverage but may also introduce documents with lower relevance.

Top-20

Top-20 returns the complete set of documents used in this experiment. It provides maximum coverage but can introduce more unrelated or less relevant results in larger collections.

9. Top-K Retrieval Results

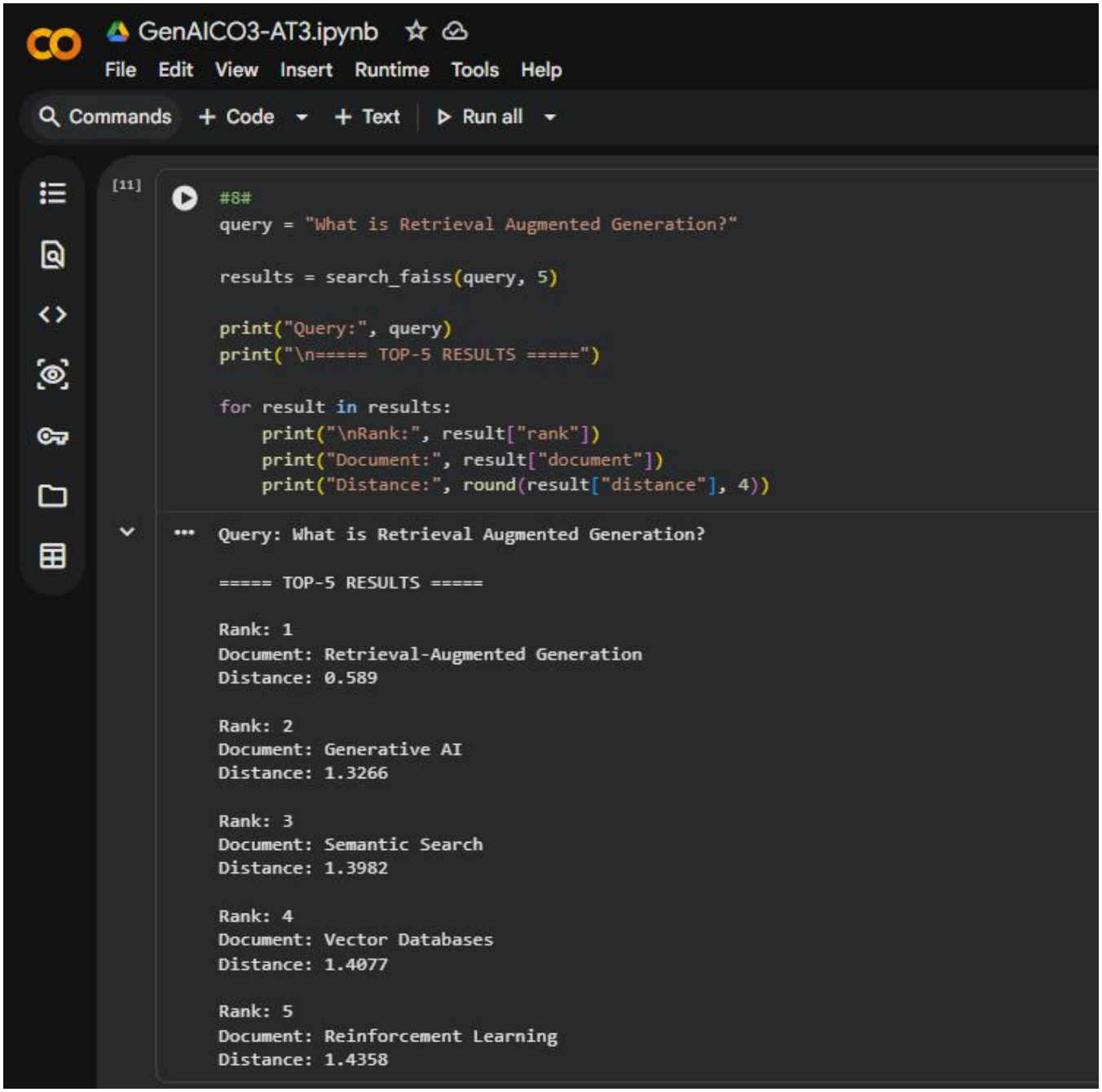
The query:

"What is Retrieval Augmented Generation?"

was used to evaluate the different Top-K values.

The results showed that the relevant Retrieval-Augmented Generation document was ranked highly, followed by semantically related documents such as Generative AI and other AI-related documents.

As K increased, the number of retrieved documents increased while the additional results became progressively broader.



10. Search Latency Analysis

Search latency was measured for Top-1, Top-3, Top-5, Top-10, and Top-20 retrieval.

The FAISS search operation was executed multiple times for each K value and the average search latency was calculated.

The measured latency values were recorded in milliseconds.

The experiment showed that FAISS provided very low search latency for all tested K values. The latency varied slightly between K values because the number of results requested from the index changed.

The exact measured values from the experiment are presented in the latency benchmark table and graph.

```python
#9#
query = "What is Retrieval Augmented Generation?"

k_values = [1, 3, 5, 10, 20]

for k in k_values:

    results = search_faiss(query, k)

    print("\n" + "=" * 70)
    print(f"TOP-{k} RESULTS")
    print("=" * 70)

    for result in results:
        print(
            f"Rank {result['rank']} | "
            f"{result['document']} | "
            f"Distance: {result['distance']:.4f}"
        )
```

```
======================================================================
TOP-1 RESULTS
======================================================================
Rank 1 | Retrieval-Augmented Generation | Distance: 0.5890

======================================================================
TOP-3 RESULTS
======================================================================
Rank 1 | Retrieval-Augmented Generation | Distance: 0.5890
Rank 2 | Generative AI | Distance: 1.3266
Rank 3 | Semantic Search | Distance: 1.3982

======================================================================
TOP-5 RESULTS
======================================================================
Rank 1 | Retrieval-Augmented Generation | Distance: 0.5890
Rank 2 | Generative AI | Distance: 1.3266
Rank 3 | Semantic Search | Distance: 1.3982
Rank 4 | Vector Databases | Distance: 1.4077
Rank 5 | Reinforcement Learning | Distance: 1.4358

======================================================================
TOP-10 RESULTS
======================================================================
```

```python
# Create query embedding once so we measure only FAISS search time
query_embedding = model.encode(
    [query],
    convert_to_numpy=True
).astype("float32")

for k in k_values:

    times = []

    # Run multiple times for a more stable average
    for _ in range(50):

        start_time = time.perf_counter()

        distances, indices = faiss_index.search(
            query_embedding,
            k
        )

        end_time = time.perf_counter()

        latency = (end_time - start_time) * 1000
        times.append(latency)

    average_latency = np.mean(times)

    latency_results.append({
        "Top-K": k,
        "Average Latency (ms)": average_latency
    })

    print(
        f"Top-{k}: "
        f"{average_latency:.6f} ms"
    )
```

```
Top-1: 0.020879 ms
Top-3: 0.012492 ms
Top-5: 0.019113 ms
Top-10: 0.012485 ms
Top-20: 0.013366 ms
```

11. Retrieval Relevance Analysis

To evaluate retrieval relevance, 10 different queries were tested.

The queries covered the main topics in the document collection, including Artificial Intelligence, Machine Learning, Deep Learning, Natural Language Processing, Computer Vision, Generative AI, Large Language Models, Retrieval-Augmented Generation, AI Agents, and AI Ethics.

The expected relevant document for each query was compared with the documents returned by FAISS.

The experiment achieved 100% Top-1 relevance accuracy for the 10 tested queries. This means the expected relevant document appeared as the first retrieved result for all the tested queries.

The relevance comparison was also performed for Top-3, Top-5, Top-10, and Top-20.

```
#11#
latency_df = pd.DataFrame(latency_results)

print(latency_df.to_string(index=False))
```

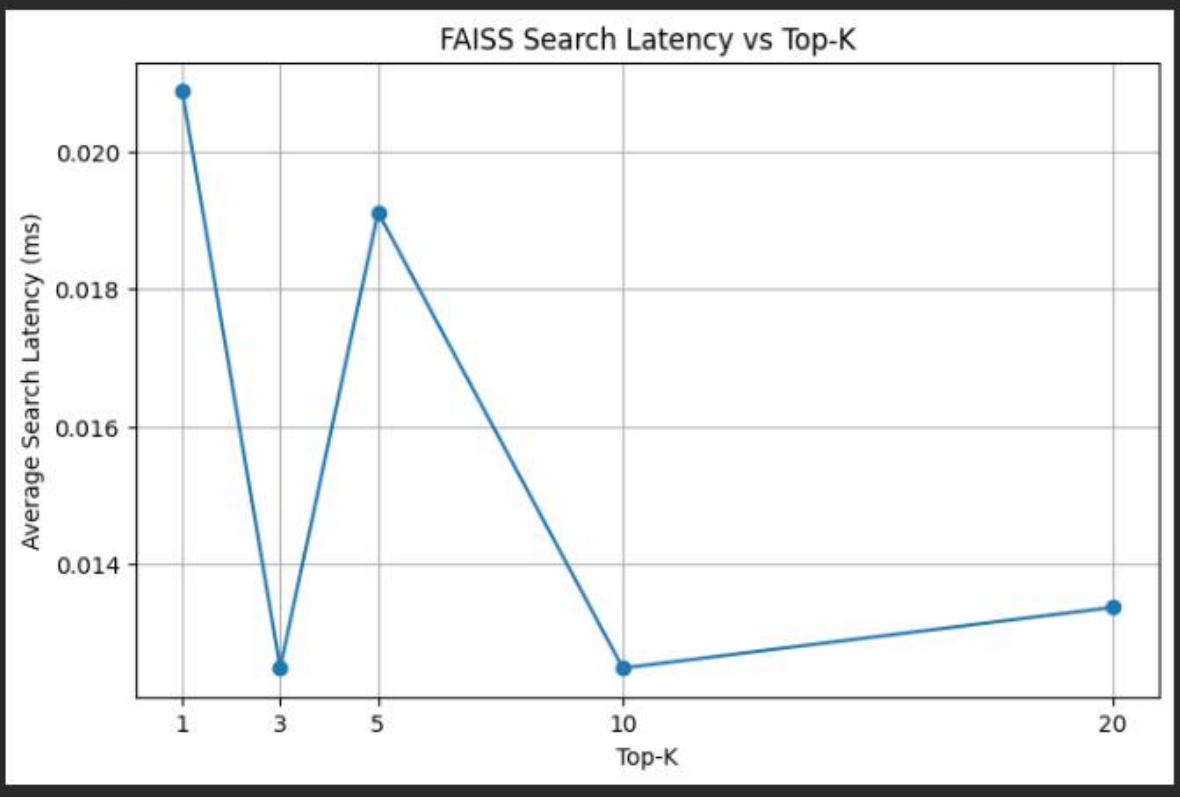
```
 Top-K  Average Latency (ms)
     1              0.020879
     3              0.012492
     5              0.019113
    10              0.012485
    20              0.013366
```

```
plt.plot(
    latency_df["Top-K"],
    latency_df["Average Latency (ms)"],
    marker="o"
)

plt.xlabel("Top-K")
plt.ylabel("Average Search Latency (ms)")
plt.title("FAISS Search Latency vs Top-K")

plt.xticks([1, 3, 5, 10, 20])
plt.grid(True)

plt.show()
```



12. Effect of Increasing K

Increasing K increases the number of documents returned by the semantic-search system.

- **Top-1** provides the most focused result.
- **Top-3** provides additional relevant context with limited retrieval overhead.
- **Top-5** provides a wider set of relevant documents.
- **Top-10** increases coverage but may include less relevant results.
- **Top-20** provides the maximum coverage in the current experiment.

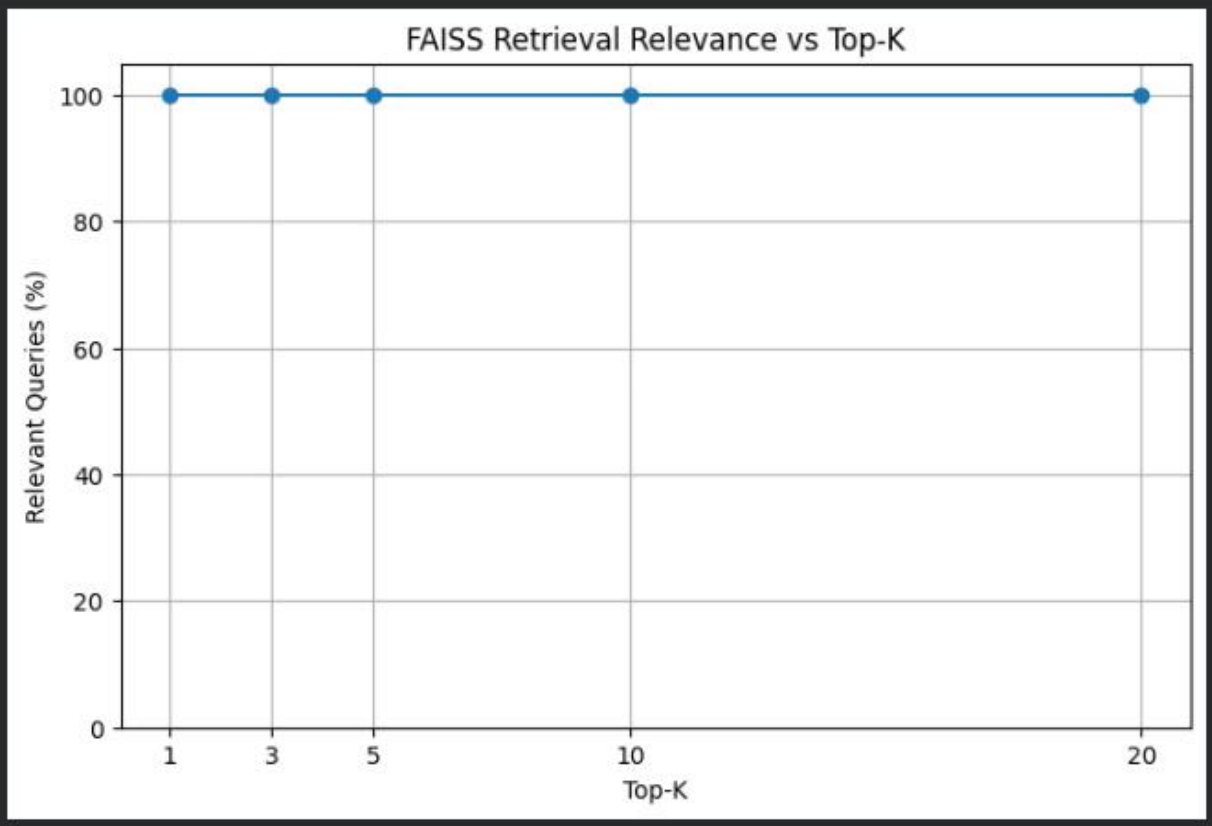
Therefore, increasing K improves information coverage, but retrieving more documents is not always necessary when the first few results are already highly relevant.

```
plt.plot(
    relevance_df["Top-K"],
    relevance_df["Relevant Queries (%)"],
    marker="o"
)

plt.xlabel("Top-K")
plt.ylabel("Relevant Queries (%)")
plt.title("FAISS Retrieval Relevance vs Top-K")

plt.xticks([1, 3, 5, 10, 20])
plt.ylim(0, 105)
plt.grid(True)

plt.show()
```



13. Latency vs Relevance Trade-off

The experiment demonstrates that K should be selected based on the application requirements.

A small K value is useful when the application requires a fast and focused response. A larger K value is useful when the application needs broader information coverage.

In this experiment, the tested queries achieved **100% relevance**, including Top-1. Therefore, increasing K did not improve the measured relevance for these queries.

This indicates that retrieving additional documents was not necessary for achieving the required relevance in this particular dataset.

14. Recommended K Value

Based on the experimental results, Top-3 is recommended as a practical K value for this semantic-search application.

Top-3 provides more context than Top-1 while maintaining a small result set and low search latency. Since Top-1 already achieved 100% relevance in the tested queries, Top-1 can also be used when only the single best result is required.

For applications requiring broader context, Top-5 can be selected.

Therefore:

Recommended K = 3

15. Result

The FAISS semantic-search system was successfully implemented and evaluated using Top-1, Top-3, Top-5, Top-10, and Top-20 retrieval.

The experiment successfully measured retrieval latency and relevance for different K values. Ten test queries were used for relevance evaluation, and the system achieved 100% Top-1 relevance accuracy.

The results demonstrate that FAISS can provide fast semantic retrieval while maintaining high relevance for the tested AI document collection.

16. Conclusion

The experiment successfully demonstrated the effect of Top-K selection on FAISS semantic-search performance.

The results showed that increasing K provides more retrieved information, but it can also increase the number of less relevant results. In the tested dataset, Top-1 already achieved 100% relevance, showing that the most relevant document could be retrieved accurately without requiring a large K value.

Considering the balance between retrieval coverage, relevance, and latency, Top-3 is recommended as a practical configuration for the implemented semantic-search application.

40. FAISS and ChromaDB are configured with the same set of documents and embedding model, but their Top-5 retrieval results differ. Design and execute a systematic debugging experiment to determine the reason for the discrepancy. Analyze the effects of distance metrics, vector normalization, query embedding generation, stored vector verification, and ranking order. Provide the experimental results and a technical explanation identifying the primary cause of the difference.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4	Level 3	Level 2	Level 1

		(Exemplary)	(Proficient)	(Developing)	(Beginning)
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

Marks Evaluation:

Question No.	Pipeline Functionality(30 %)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	